



Hochschule
Bonn-Rhein-Sieg
University of Applied Sciences



DLRV Project

Flow-Based Label Propagation for UAV Video Segmentation

Sunesh Praveen Raja Sundarasami

Submitted to Hochschule Bonn-Rhein-Sieg,
Department of Computer Science
in partial fulfilment of the requirements of the
Deep Learning for Robot Vision course

Supervised by

Prof. Dr. Sebastian Houben

September 2026

I, the undersigned below, declare that this work has not previously been submitted to this or any other university and that it is, unless otherwise stated, entirely my own work. The report was, in part, written with the help of the AI assistant Claude (Anthropic) as described in the appendix. I am aware that content generated by AI systems is no substitute for careful scientific work, which is why all AI-generated content has been critically reviewed by me, and I take full responsibility for it.

Date

Sunesh Praveen Raja Sundarasami

Abstract

Segmenting every frame of a UAV video with a per-frame model is expensive: the strongest model trained here costs 370 ms per 4 K frame, so a 45 s clip at 20 FPS takes 5.6 minutes of GPU time. Consecutive frames are almost identical, so most of that computation repeats work already done.

This report evaluates an *adaptive hybrid* alternative: segment one frame with a model, propagate the mask to later frames by warping it along optical flow, and re-run the model only when the propagated mask becomes unreliable. Reliability is measured by a forward-backward consistency check, accumulated across steps into a per-frame *coverage* value; the model is re-invoked when coverage falls to a threshold τ , which resets coverage. Components were selected by measurement: of the backbones trained behind one UPerNet decoder, ConvNeXt-Large is strongest at 0.8166 mIoU, and SEA-RAFT propagates as accurately as FlowNet2 at $3.6\times$ the speed.

On all 7 UAVid validation sequences (6,307 frames), the pipeline is $1.65\times$ faster than a model-every-frame baseline, retains 96.2% of its mIoU, and calls the model on 19.7% of frames under one fixed threshold. Because flow is computed for every consecutive pair before the threshold can be tested, flow is the fixed cost and model inference the marginal one, so τ controls quality much more than speed. The residual loss concentrates on small moving objects, where motion breaks the consistency check. The source code is available at <https://git.inf.h-brs.de/ssunda2s/flow-based-label-propagation-for-uav-video-segmentation>.

Acknowledgements

I thank Prof. Dr. Sebastian Houben for supervising this project.

An AI assistant was used during this project; its role is described in Appendix B.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Challenges	1
1.3	Problem Statement	2
1.4	Contributions and Scope	2
2	State of the Art	3
2.1	Aerial Video Segmentation Datasets	3
2.2	Semantic Segmentation Architectures	3
2.3	Optical Flow Estimation	3
2.4	Flow-Based Label Propagation and the Gap	4
3	Methodology	5
3.1	Datasets and Label Scheme	5
3.2	Quality Metric	5
3.3	Timing and Experimental Design	6
4	Solution	7
4.1	System Overview	7
4.2	Segmentation Models	7
4.3	Optical Flow Backends	7
4.4	Propagation Primitives	9
4.4.1	Backward Warp	9
4.4.2	Forward-Backward Consistency	9
4.4.3	Coverage Accumulation	10
4.5	The Adaptive Algorithm and Implementation	11
5	Evaluation and Results	13
5.1	Segmentation Backbone	13
5.2	Optical Flow Backend	13
5.3	What the Coverage Threshold Controls	14
5.4	Validation Split Results	14
5.5	Where Quality Is Lost	16
5.6	Threats to Validity	16

6 Conclusions 17

6.1 Contributions 17

6.2 Lessons Learned 17

6.3 Future Work 18

Appendix A Pipeline Configuration 19

Appendix B Description of AI-Generated Content 21

References 23

1

Introduction

1.1 Motivation

A drone that has to act on what it sees needs a semantic label for every frame it captures. Obstacle avoidance, landing-site selection and terrain following all read from a dense map of the scene, and they read from it while the aircraft is still moving.

Producing that map one frame at a time is expensive. The strongest segmentation model trained for this project takes about 370 ms on a native 4K frame, so labelling a 45 s clip costs 5.6 minutes of GPU time, roughly $7.4\times$ longer than the flight it describes, and on an NVIDIA A100 rather than on anything a drone could carry. Chapter 5 reports the model and the measurement in full.

Much of that work is also repeated. At 20FPS consecutive frames differ by a few tens of pixels of apparent motion, so the network recomputes almost the same mask twenty times a second. If the mask from one frame could be carried forward to the next, the model would need to run only when carrying it forward stops working.

That is the system evaluated here: segment one frame, move its mask onto the frames that follow along optical flow, and re-run the model only when the moved mask can no longer be trusted. Optical flow supplies the correspondence, since a flow field states where each pixel went and a label attached to that pixel can travel with it, and learned estimators such as RAFT [1] and SEA-RAFT [2] are accurate enough for this to be done directly. The experiments use UAVid [3], an urban-drone benchmark of oblique, low-altitude video.

Carrying labels along flow is not itself new; what differs here is where the first mask comes from. SegProp [4] and the video propagation of Zhu et al. [5] propagate *human-drawn* annotations offline, to grow a training set from a few labelled frames. In flight there is no annotation to propagate, so the mask must be a model prediction produced on the fly. That makes producing it the cost worth avoiding, and it forces the system to judge for itself when the propagated mask has decayed enough to be worth replacing.

1.2 Challenges

Flow is the fixed cost. RAFT-family models [1,2] build an all-pairs correlation volume whose size grows *quadratically* with resolution, so at 4K their memory is dominated by that activation rather than

by weights. More importantly, flow must be computed for every consecutive pair before the staleness test can be evaluated, so it is paid whether the model runs or not. This bounds the achievable speedup.

Detecting bad flow is harder than computing it. Flow fails at occlusion boundaries, on fast-moving objects and under parallax, and the propagated label is then wrong. The standard detector is the forward-backward consistency check [6], which rejects a pixel whose round trip through both flow directions does not return to its origin. It tests whether the two flow fields *agree*, not whether the label is *correct*.

Ground truth is sparse. UAVid labels one frame in a hundred [3], so every quality figure here rests on 70 evaluation points across 7 sequences: enough for a paired comparison of two methods on identical frames, not enough to characterise the distribution of propagation quality.

1.3 Problem Statement

Given a UAV video $\{I_0, \dots, I_T\}$, a segmentation model f costing t_{model} per frame and a flow model costing t_{flow} per frame pair, produce a dense mask for *every* frame in less than $T \cdot t_{\text{model}}$ wall-clock time, with mean quality as close as possible to running f on every frame.

The policy evaluated is: run f on frame 0; propagate its output along flow; maintain a cumulative estimate γ of the fraction of the frame still trustworthy; and re-invoke f when γ falls to a threshold τ . Three questions are asked of it: how much time it saves, how much quality that costs, and what τ controls.

1.4 Contributions and Scope

1. **An adaptive hybrid inference pipeline** that propagates model predictions along SEA-RAFT flow and re-invokes the model on a forward-backward coverage trigger, running $1.65\times$ faster than model-every-frame inference at 96.2% of its mIoU on the UAVid validation split (Section 5.4).
2. **A component study** selecting the segmentation backbone (Section 5.1) and the flow backend (Section 5.2) by measurement.
3. **A characterisation of the coverage threshold** as a quality control rather than a speed control, which follows from flow being the fixed cost (Section 5.3).
4. **A reproducible implementation** with interchangeable backbones and flow backends, chunked streaming flow that keeps peak memory flat regardless of video length, and a resolution setting that makes 4 K flow tractable (Chapter 4).

The scope is narrow. Evaluation uses the UAVid validation split only, since the test labels are withheld [3]; the flow-backend comparison uses Ruralscapes for the reason given in Section 5.2. Each backbone was trained once with one seed, so backbone differences support a ranking and not a significance claim. Propagated masks are used unrefined, and there is no deployment engineering: no quantisation, distillation or TensorRT export, and all timings come from one A100 under bf16 PyTorch.

State of the Art

2.1 Aerial Video Segmentation Datasets

UAVid [3] is an urban-drone dataset of oblique 4 K video, labelled densely at intervals rather than continuously. Among its stated challenges are large scale variation, moving-object recognition and temporal consistency across frames; the last is the property flow propagation exploits, which is why it is the primary dataset here.

Ruralscapes, introduced with SegProp [4], is a 4 K aerial dataset of rural scenes whose dense labels come twice as often as UAVid’s, every 50 frames. That is what makes it usable for the single-jump propagation experiments of Section 5.2.

2.2 Semantic Segmentation Architectures

The models here follow the standard encoder-decoder pattern for dense prediction: a backbone reduces the image to a multi-scale feature pyramid, and a decoder fuses those scales into per-pixel logits.

The decoder throughout is **UPerNet** [7], which combines the Pyramid Pooling Module of **PSPNet** [8], adaptive average pooling at several grid resolutions, with a Feature Pyramid Network top-down fusion path. It is backbone-agnostic, so encoders swap without changing the head.

Four encoder families were considered. **ConvNeXt** [9] modernises a ResNet toward transformer design choices while remaining convolutional, and is strongest here. **Swin** [10] is a hierarchical transformer built from shifted local-window attention, linear in image size. **SegFormer** [11] pairs a hierarchical transformer encoder with a minimal all-MLP decoder. **Hiera** [12] removes architectural extras from a hierarchical vision transformer and relies on masked-autoencoder pretraining instead; it is the image encoder of **SAM 2** [13], whose public checkpoints supply an initialisation at no training cost.

2.3 Optical Flow Estimation

Optical flow assigns to each pixel of one frame a displacement to its corresponding location in another. Four steps of that lineage matter here.

Variational methods. Brox et al. [14] formulate flow as energy minimisation over brightness constancy, gradient constancy and a discontinuity-preserving smoothness term, solved by nested fixed-point iterations implementing coarse-to-fine warping. This is the classical reference for the warping principle mask propagation rests on.

Occlusion detection by round trip. Following a pixel forward and then back should return it near its origin; if it does not, the correspondence is unreliable. Sundaram, Brox and Keutzer [6] use this forward-backward criterion to filter point trajectories, and it is the direct ancestor of the validity check used here.¹

Learned flow. FlowNet [15] showed that end-to-end CNN flow estimation was viable. **FlowNet2** [16] made it competitive by stacking FlowNetC, FlowNetS and FlowNetSD into a cascade evaluated in one forward pass; it is the backend SegProp uses, which is why it is retained here for comparison.

Iterative refinement. **RAFT** [1] extracts per-pixel features, builds an all-pairs 4D correlation volume, and refines a flow field with a recurrent unit that looks up that volume. **SEA-RAFT** [2] simplifies and accelerates RAFT while improving accuracy, reaching 3.69 px endpoint error on Spring, a 22.9 % error reduction over prior published results. It is the default backend here, though its correlation volume scales quadratically with resolution and becomes the dominant engineering constraint at 4 K (Section 4.3).

2.4 Flow-Based Label Propagation and the Gap

SegProp [4] is the closest prior work. Annotators label roughly 2 % of a video’s frames and SegProp propagates those labels to the rest along FlowNet2 flow by iterative class voting; the propagated labels then train a segmentation network that outperforms its fully supervised counterpart. Keyframes are selected at fixed intervals, independently of scene content.

Two gaps follow, and together they define this project. The first is *where the starting mask comes from*. Prior work starts from a frame a human has labelled, which is possible offline but not in flight. Here the starting mask has to be a model prediction, so it is imperfect to begin with, it is the expensive part of the system, and something has to decide when to pay for it. The second is *when to compute a new one*. Picking keyframes at a fixed interval ignores what the camera is doing: one interval wastes model calls during a hover and falls behind during fast motion. A trigger that watches the scene instead is the alternative, and the forward-backward check costs nothing extra, because propagation has already computed both flow directions. Prior work also measures label quality rather than time, and time is what decides whether propagating is worth doing at all.

¹The criterion is often attributed to Brox et al. [14], which introduces the variational warping formulation; the forward-backward occlusion test appears in the later trajectory work [6].

3

Methodology

3.1 Datasets and Label Scheme

All adaptive-inference experiments use the UAVid v1.5 release [3], whose properties are listed in Table 3.1. Ruralscapes is used only for the flow-backend comparison of Section 5.2.

Training uses the ten labelled keyframes of each of the 20 training sequences, 200 images. The adaptive pipeline decodes each full video sequentially and is scored wherever a labelled frame exists.

UAVid’s colour-coded label images are decoded to six classes plus an ignore class (Table 3.2). Two consequences matter downstream: moving and static cars are *merged* into one **vehicle** class, and background becomes **ignore**, excluded from the loss and from every mIoU reported here.

3.2 Quality Metric

Quality is mean intersection-over-union, from a confusion matrix accumulated over all non-ignored pixels:

$$\text{IoU}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c + \text{FN}_c}, \quad \text{mIoU} = \frac{1}{|\mathcal{C}_{\text{gt}}|} \sum_{c \in \mathcal{C}_{\text{gt}}} \text{IoU}_c, \quad (3.1)$$

where $\mathcal{C}_{\text{gt}}$ is the set of classes present in that frame’s ground truth.

Equal class weighting. The mean is unweighted, so a class occupying a few hundred pixels counts as much as one occupying a million. **person** regions in UAVid are small, so a few misassigned boundary pixels cost a large *fraction* of that class’s IoU and move the aggregate mIoU as much as a far larger error on **building** would. This explains most of the degradation pattern of Section 5.5.

Two masking modes. mIoU is computed over two different sets of pixels. The *all-pixel* figure scores every pixel that is not **ignore**. The *valid-only* figure also leaves out the pixels the forward-backward check has rejected, so it measures the warp only where flow is trustworthy. Every hybrid-against-baseline comparison uses the all-pixel figure, so both methods are judged on exactly the same pixels; valid-only numbers appear only in the flow-backend comparison.

Table 3.1: UAVid v1.5 properties, verified from the release on disk by file inspection.

Property	Value
Sequences	42 (20 train / 7 validation / 15 test)
Frames per sequence	901, exactly
Frame rate	20 FPS, exactly
Native resolution	3840×2160 (4 K)
Densely labelled frames	ten per sequence, one every 100 frames

Table 3.2: The six-class scheme used throughout.

Id	Class	Note
0	road	
1	vegetation	low vegetation
2	tree	
3	person	smallest, rarest object
4	vehicle	moving and static car, merged
5	building	
255	ignore	excluded from loss and mIoU

One caveat across chapters. Training-time validation mIoU (Table 5.1) is computed on a 1024^2 *centre crop*, the training pipeline’s validation transform, whereas the pipeline experiments score full-resolution frames. The two are not the same measurement, and the gap between ConvNeXt-Large’s 0.8166 and the pipeline baseline’s 0.7851 is mostly that difference rather than a degraded baseline.

3.3 Timing and Experimental Design

Every timing is wall-clock and is taken with the GPU synchronised, so no work can hide behind an asynchronous kernel launch. Each frame is charged what the pipeline spends on it: model time on a model frame, and flow, warp and consistency-check time on a propagated one. Baseline and hybrid run over the same video on the same device, so the comparison is paired.

All experiments ran on a single NVIDIA A100 80 GB under PyTorch 2.6 with CUDA 12.4, in `bfloat16` at native resolution, with the SEA-RAFT `spring-L` checkpoint at 12 refinement iterations.

Four experiments follow: the backbone comparison (Section 5.1), the flow-backend comparison (Section 5.2), a sweep of τ over $\{0.75, 0.90, 0.95, 1.00\}$ (Section 5.3), and the full validation split at $\tau = 0.95$ (Section 5.4). Quality is measured at the 70 dense frames and timing over all 6,307; Section 5.6 states the limits of this protocol.

4

Solution

4.1 System Overview

The system has three parts: a segmentation model that labels any frame accurately but expensively, an optical-flow model that moves an existing mask to the next frame cheaply, and a controller that chooses between them (Figure 4.1).

4.2 Segmentation Models

Every model shares one shape: an encoder producing a four-level feature pyramid at strides 4, 8, 16 and 32, and a UPerNet decoder [7] fusing those levels into per-pixel logits, bilinearly upsampled to the input resolution (Figure 4.2). Every wrapper enforces the same contract, input $(B, 3, H, W)$ and output (B, C, H, W) , which makes the backbones interchangeable. Backbones start pretrained, from ImageNet-derived UPerNet weights for ConvNeXt and Swin and from SAM 2 [13] for Hierarchical, and only the six-class classifier trains from scratch, so convergence takes tens rather than hundreds of epochs.

The Hierarchical variants [12] needed one fix to run at native resolution. Hierarchical combines an interpolated global positional embedding with a windowed embedding tiled by exact integer repetition. Training at 1024^2 gives a 256×256 feature map, an exact multiple of the window size, so the code worked; 4K input gives 960×540 , and $540/8 = 67.5$ is not integral, so it failed unconditionally. Replacing the exact tile with a ceiling-tile-then-crop reproduces the old output wherever it was defined, so no retraining was needed.

One training recipe serves every backbone, varying only batch size with memory footprint (Table 4.1). Random scale augmentation matters here because UAVid is captured at roughly constant altitude and so contains little natural scale variation.

4.3 Optical Flow Backends

Both backends satisfy one interface: estimate flow for a pair, for a batch of pairs, or for a batch returning a GPU-resident tensor, the last of which lets the pipeline warp and score without a host round trip. Flow is returned in pixels (Figure 4.4).

SEA-RAFT [2] follows the RAFT recipe (Figure 4.5): encode features at $1/8$ resolution, build an all-pairs correlation volume, regress an initial flow field, refine it for a fixed number of recurrent iterations that look up the volume at the current estimate, then convex upsample to full resolution. The configuration

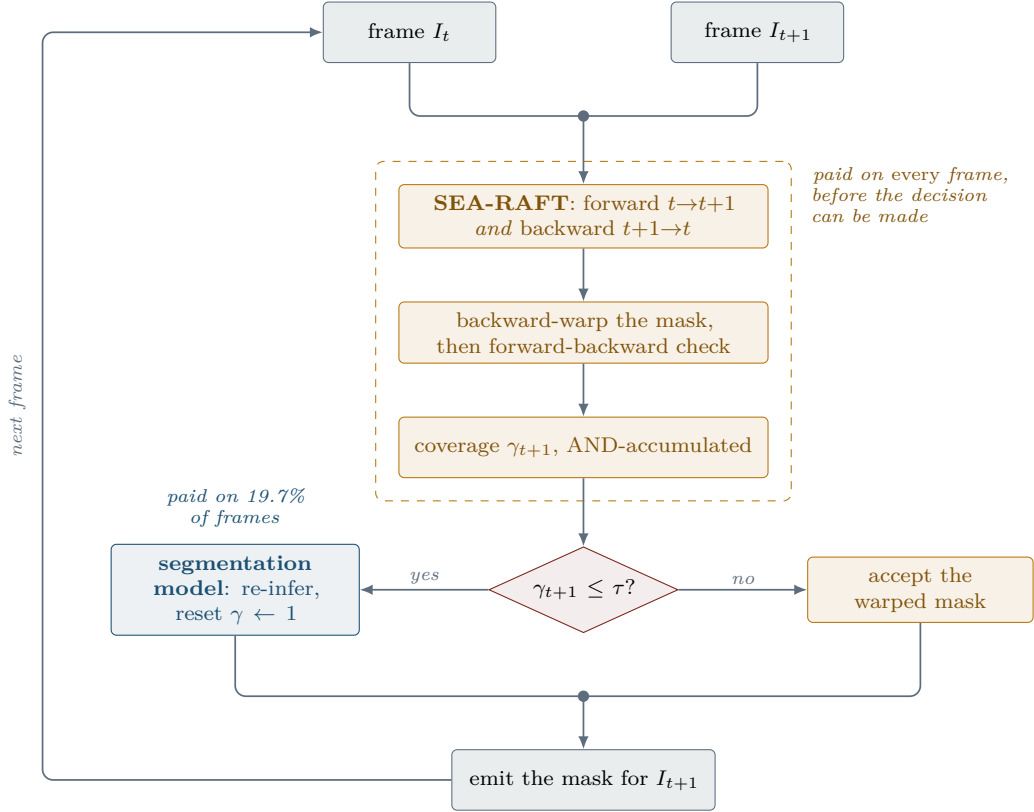


Figure 4.1: The adaptive hybrid pipeline. Everything in the dashed box runs for each consecutive frame pair *before* the threshold test, so its cost is unconditional; only the model call is conditional. That asymmetry, not the threshold, bounds the achievable speedup (Section 5.3).

used is the **spring-L** checkpoint with 12 refinement iterations, a ResNet-34 feature encoder, correlation radius 4 and feature dimension 128.

The correlation volume dominates cost. For a frame of height H and width W its size grows as $(H/8 \cdot W/8)^2$, so at 4 K a single pair needs roughly 20 GB, even though SEA-RAFT has far fewer *parameters* than the segmentation model. Parameter count is a poor proxy for cost here.

The remedy is a **scale** parameter, which runs the network at a lower internal resolution and rescales the output vectors back to full-resolution pixels. This project uses quarter resolution, **scale**=−2. Flow time falls by more than a factor of three, and a hard cuDNN limit that aborts full-resolution batches whatever the free memory is avoided. The flow is still returned at full input resolution, so mask resolution is unaffected, and Section 5.2 measures the accuracy cost at under 0.01 mIoU.

FlowNet2 [16] is retained for comparison because SegProp uses it. It is a single-pass cascade with no iterative refinement and no equivalent resolution control, and it needs custom CUDA operators to run at all, which is part of why SEA-RAFT is the default.

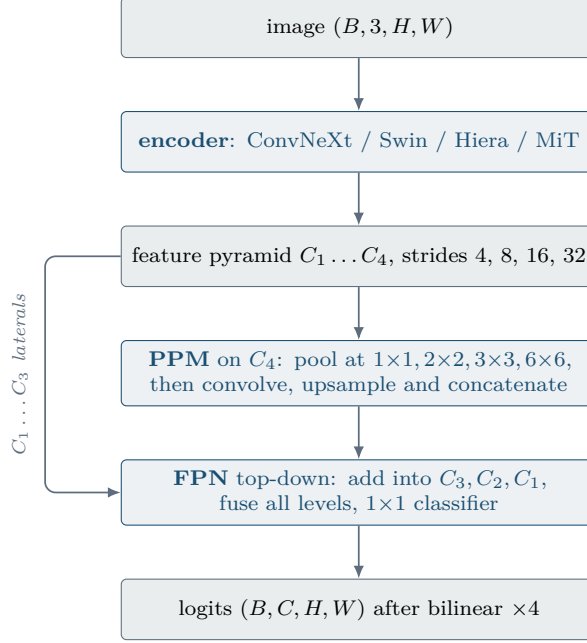


Figure 4.2: The architecture shared by every trained model: an encoder, a four-level pyramid, and a UPerNet decoder combining pyramid pooling [8] for global context with top-down FPN fusion for spatial detail.

4.4 Propagation Primitives

Three operations, applied to whole GPU-resident batches so that no per-frame work returns to the host.

4.4.1 Backward Warp

To label target frame $t+1$ from a mask at t , the required flow is *backward*, from target to source. For each target pixel (x, y) , $\mathbf{f}_{\text{bwd}}(x, y) = (u, v)$ says the content came from $(x+u, y+v)$ in the source, so the target label is read from the source mask at that location (Figure 4.3a). Sampling is **nearest-neighbour** rather than bilinear, since labels are categorical and interpolating class 2 with class 5 yields either a meaningless value or a valid but wrong class. Target pixels whose source falls outside the frame become **ignore**. Backward warping is preferred to forward splatting because it gives every target pixel exactly one source, leaving no holes and no collisions.

4.4.2 Forward-Backward Consistency

Warped labels inherit the flow’s errors. The forward-backward check [6, 14] detects unreliable correspondences by round trip (Figure 4.3b): step from target pixel p to its claimed source q along backward flow, step from q along forward flow sampled bilinearly at that sub-pixel location, and measure the distance

Table 4.1: Training configuration, shared by every backbone except for batch size, which follows each model’s memory footprint.

Setting	Value
Optimiser	AdamW [17], lr 10^{-4} , weight decay 0.05
Schedule	cosine, with 2-epoch linear warmup
Stopping	early stop on a validation-mIoU plateau
Input	1024^2 random crops, <code>bfloat16</code> AMP
Loss	cross-entropy, ignoring class 255
Augmentation	horizontal flip; random scale 0.75 to 1.4; brightness and contrast; hue and saturation
Batch size	8 (ConvNeXt-Large), 4 (Swin-Large and Hiera-Base+)



(a) backward warp: one source per target pixel

(b) FB check: valid iff $\epsilon < 1.5$ pxFigure 4.3: The two propagation primitives. (a) Backward flow at target pixel p names its source q , whose label is copied to p by nearest-neighbour sampling. (b) A round trip through both flow directions returns to r ; the distance ϵ from the original p is the consistency error of Equation 4.1.

from p :

$$\epsilon(p) = \|\mathbf{f}_{\text{bwd}}(p) + \mathbf{f}_{\text{fwd}}(p + \mathbf{f}_{\text{bwd}}(p))\|_2. \quad (4.1)$$

A pixel is valid when $\epsilon(p) < \theta$ and its source stayed in bounds, with $\theta = 1.5$ px throughout.

What the check cannot see. It tests whether the two flow fields agree geometrically. It is blind to the nearest-neighbour rounding introduced at every warp step, which accumulates at object boundaries while flow stays self-consistent. This is why a frame can report high coverage and still carry degraded labels on small moving objects (Section 5.5).

4.4.3 Coverage Accumulation

The check in Section 4.4.2 grades one warp. For the step from frame t to frame $t+1$ it marks every pixel true or false; call that map S_{t+1} . On its own it says nothing about history: a pixel can pass this step even though it was warped badly three frames ago, because the labels it carries came from there.

The pipeline therefore keeps a second map, V_t , recording which pixels of frame t have passed *every* step since the last model call. It is set all-true immediately after inference. Updating it takes two operations. Since V_t is expressed in frame t ’s pixel grid, it is first warped into frame $t+1$ with the same backward flow

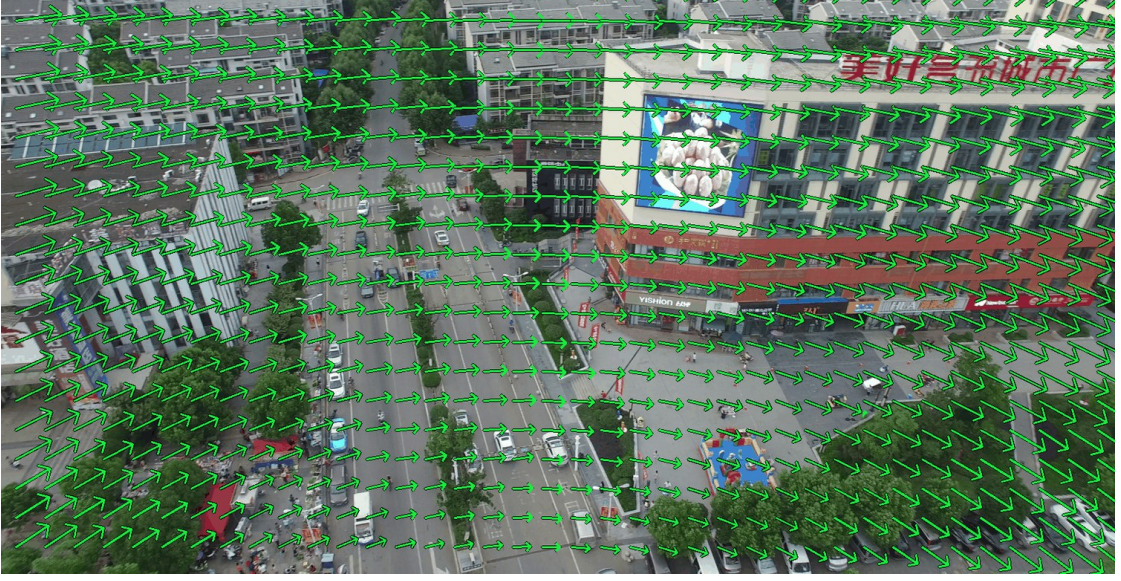


Figure 4.4: SEA-RAFT flow on the fastest-motion pair of a UAVid sequence, subsampled to one arrow per grid cell. Each arrow is the estimated displacement into the next frame, and labels are propagated along these vectors. The field is dominated by camera motion; the disagreement remaining at object boundaries is what the forward-backward check is meant to catch.

used for the labels, then combined with the new step’s map by a per-pixel AND:

$$V_{t+1} = \text{warp}(V_t) \wedge S_{t+1}, \quad \gamma_{t+1} = \frac{1}{HW} \sum V_{t+1}. \quad (4.2)$$

Coverage γ_{t+1} is the fraction of the frame’s HW pixels that are still true, and that single number is what the controller compares against τ . A pixel marked false can never become true again, so γ only falls while labels are propagated, and returns to 1 at the next model call.

4.5 The Adaptive Algorithm and Implementation

Algorithm 1 states the policy. Flow is computed for the whole chunk at line 5, before the threshold test at line 10, so every frame pays for flow whether or not the model then runs.

Flow cannot be computed up front: the forward and backward fields for a 900-frame 4K video would occupy about 120 GB. The pipeline therefore streams, computing flow in windows of $K = 16$ steps and discarding each window once its frames are labelled, so memory stays flat however long the video is. Larger windows proved unreliable once a segmentation model shared the GPU.

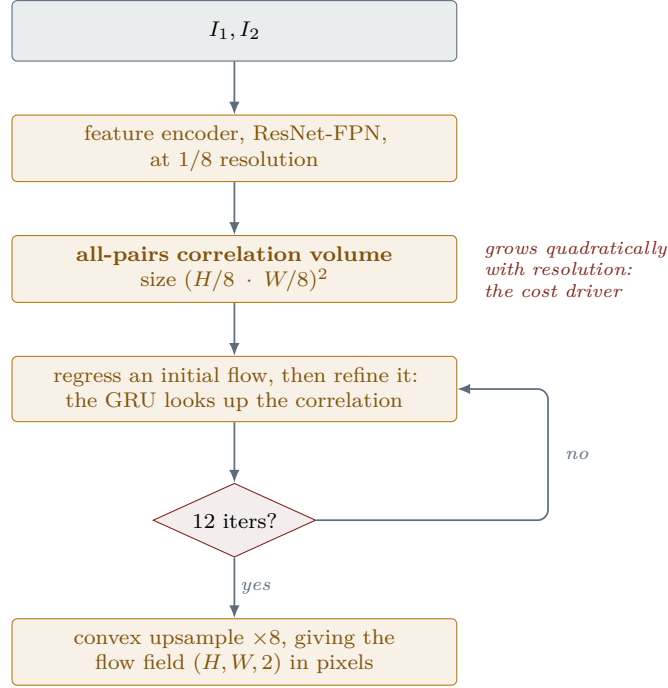


Figure 4.5: SEA-RAFT’s refinement loop [1,2]. The correlation volume grows quadratically with resolution, so the internal working resolution is the dominant cost setting at 4 K.

Algorithm 1: Adaptive hybrid inference

input : frames $I_0..I_T$; model f ; flow model g ; threshold τ ; chunk size K
output : a dense mask M_t for every frame

```

1  $M_0 \leftarrow f(I_0)$ ;  $V_0 \leftarrow \mathbf{1}$ ; emit ( $M_0$ , “model”,  $\gamma=1$ );
2  $s \leftarrow 0$ ;
3 while  $s < T$  do
4    $E \leftarrow \min(s + K, T)$ ;
5    $\{\mathbf{f}_i^{\text{fwd}}\}, \{\mathbf{f}_i^{\text{bwd}}\} \leftarrow g(\{(I_i, I_{i+1})\}_{i=s}^{E-1})$ ; // one batched launch
6   for  $i \leftarrow s$  to  $E - 1$  do // nearest neighbour
7      $M' \leftarrow \text{warp}(M_i, \mathbf{f}_i^{\text{bwd}})$ ;
8      $S \leftarrow \text{fbcheck}(\mathbf{f}_i^{\text{fwd}}, \mathbf{f}_i^{\text{bwd}}, \theta)$ ;
9      $V' \leftarrow \text{warp}(V_i, \mathbf{f}_i^{\text{bwd}}) \wedge S$ ;  $\gamma \leftarrow \text{mean}(V')$ ;
10    if  $\gamma \leq \tau$  then
11       $M_{i+1} \leftarrow f(I_{i+1})$ ;  $V_{i+1} \leftarrow \mathbf{1}$ ;
12      emit ( $M_{i+1}$ , “model”,  $\gamma=1$ );
13    else
14       $M_{i+1} \leftarrow M'$ ;  $V_{i+1} \leftarrow V'$ ;
15      emit ( $M_{i+1}$ , “flow”,  $\gamma$ );
16  discard chunk flow;  $s \leftarrow E$ ;
  
```

5

Evaluation and Results

The four sections follow the four experiments of Section 3.3: choose the backbone, choose the flow backend, characterise the coverage threshold, then run the full validation split.

5.1 Segmentation Backbone

The encoders were trained behind the same UPerNet decoder under the recipe of Table 4.1; Table 5.1 compares them.

ConvNeXt-Large has both the highest mIoU and the lowest latency, so the pipeline uses it in every subsequent experiment. Swin-Large is within 0.009 mIoU but 68 ms slower per frame, and Hiera-Base+ trades 0.036 mIoU for a 57 % smaller parameter count (Figure 5.1).

5.2 Optical Flow Backend

SEA-RAFT and FlowNet2 were compared on 90 matched keyframe-to-target propagation pairs scored against dense ground truth (Table 5.2).

The backends are indistinguishable on quality: propagation mIoU differs by 0.0006, valid-pixel mIoU by 0.0001 and coverage by 0.4 %. SEA-RAFT is $3.6\times$ faster per pair and simpler to deploy, so the choice rests on speed and maintainability rather than accuracy. The result also confirms that `scale=-2` (Section 4.3) costs under 0.01 mIoU while removing more than two thirds of the flow time.

Why this experiment uses Ruralscapes. It inherited a single-jump propagation harness built for that dataset and was not re-run on UAVid. The regime therefore differs from the pipeline’s, one flow

Table 5.1: Trained segmentation models. Validation mIoU is at each model’s best epoch on the 1024^2 centre-crop validation transform (Section 3.2).

Model	Params	Inference	Val mIoU
ConvNeXt-Large	234.9 M	365 ms	0.8166
Swin-Large	233.9 M	433 ms	0.8081
Hiera-Base+	101.0 M	327 ms	0.7802

Inference is a single-frame 4K measurement on the A100. That run wrote no persistent log, so unlike every other number in this chapter these three latencies come from project notes.

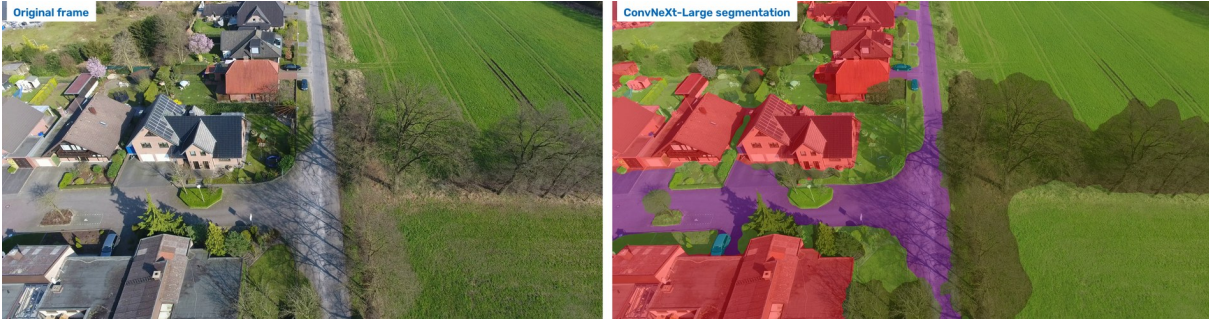


Figure 5.1: ConvNeXt-Large on seq36 frame 100: buildings in red, road in purple, trees and vegetation in green. The frame scores 0.917 mIoU against its dense ground truth.

estimate spanning 50 frames rather than 50 chained consecutive estimates, and Ruralscapes has twelve classes against UAVid’s six, so these absolute mIoU values are not comparable with Section 5.4. The experiment is used only for the *relative* ranking of two backends on identical pairs.

5.3 What the Coverage Threshold Controls

The threshold τ was swept over $\{0.75, 0.90, 0.95, 1.00\}$ on seq16 with ConvNeXt-Large and SEA-RAFT held fixed (Table 5.3).

The threshold controls quality, not speed. This follows from the cost structure of Figure 4.1. Dropping from 901 model calls to 71 removes 92% of the inference work but yields only $1.9\times$, because flow runs on every frame regardless; going further, to 5 calls, moves only to $2.2\times$ while costing 0.20 mIoU. Equivalently, raising τ from 0.75 to 1.00 lifts mIoU from 0.51 to 0.76 for under 15% more runtime. There is therefore no reason to operate at a low threshold, and $\tau = 0.95$ is used throughout.

5.4 Validation Split Results

Baseline and hybrid were run over all 7 UAVid validation sequences, 6,307 frames, at $\tau = 0.95$ (Table 5.4).

Table 5.2: SEA-RAFT against FlowNet2 on 90 matched Ruralscapes propagation pairs. Propagation mIoU is the all-pixel figure, the valid-pixel row excludes pixels the FB check rejects, and coverage is the mean fraction of pixels it accepts.

	SEA-RAFT (default)	FlowNet2
Inference per pair (4K)	150 ms	532 ms
Throughput	6.7 pairs/s	1.9 pairs/s
Propagation mIoU	0.7957	0.7951
mIoU on valid pixels	0.8162	0.8160
Valid coverage	88.4%	88.8%

The per-pair timings are development measurements whose log was not retained; the quality and coverage figures are re-derived from the committed per-pair metrics files.

5. Evaluation and Results

Table 5.3: Threshold sweep on seq16, 901 frames, ConvNeXt-Large with SEA-RAFT. $\tau = 1.00$ degenerates to the model-every-frame baseline, since coverage can never exceed 1.

Threshold τ	Model calls	Speed vs. every frame	mIoU
0.75	5	$2.2\times$	0.51
0.90	23	$2.1\times$	0.62
0.95 (default)	71	$1.9\times$	0.71
1.00 (baseline)	901	$1.0\times$	0.76

The $\tau = 0.75$ and $\tau = 0.90$ rows are development runs whose per-frame logs were not retained.

Table 5.4: Baseline against adaptive hybrid on all 7 UAVid validation sequences, $\tau = 0.95$, ConvNeXt-Large with SEA-RAFT. Times are wall-clock over all 901 frames of each sequence; mIoU is the mean over that sequence’s ten dense ground-truth frames, scored on identical pixel sets for both methods.

Sequence	Model calls	Baseline	Hybrid	Speedup	Base mIoU	Hyb. mIoU
seq16	71	337 s	181 s	$1.86\times$	0.763	0.711
seq17	208	334 s	210 s	$1.59\times$	0.780	0.758
seq18	76	334 s	184 s	$1.82\times$	0.835	0.813
seq19	194	364 s	222 s	$1.64\times$	0.744	0.728
seq20	474	363 s	279 s	$1.30\times$	0.773	0.739
seq36	131	365 s	211 s	$1.73\times$	0.842	0.831
seq37	87	335 s	189 s	$1.77\times$	0.759	0.708
All	177	2433 s	1476 s	$1.65\times$	0.785	0.755

The hybrid is $1.65\times$ faster than model-every-frame inference, retains 96.2% of the baseline mIoU (0.7554 against 0.7851), and calls the model on 19.7% of frames, 1,241 calls across 6,307 frames.

Model calls follow motion, without per-sequence tuning. Under one fixed threshold the high-motion seq20 uses 474 model calls and the calm seq37 uses 87. At that extreme the speedup falls to $1.30\times$, the lowest in the table, but quality holds at 0.739 against a 0.773 baseline: the cost of motion is paid in speed rather than accuracy. On the propagated frame of Figure 5.2 the gap to the baseline is 0.005 mIoU.



Figure 5.2: Left: ConvNeXt-Large on every frame. Right: the adaptive hybrid at $\tau = 0.95$ on the same frame, labelled by propagation rather than inference (`src=flow`) at 97% coverage. The mIoU difference is 0.005.

5.5 Where Quality Is Lost

Propagation costs 0.030 mIoU in aggregate, from 0.7851 to 0.7554, concentrated on small moving objects, principally **person** and **vehicle**. Three properties combine to produce this. First, mIoU averages over classes without weighting by pixel count (Section 3.2), so a **person** region of a few hundred pixels loses a large *fraction* of its IoU to a boundary ring a few pixels wide. Second, every step resamples with nearest-neighbour interpolation, adding a rounding error at every boundary that accumulates along a run, invisible to the consistency check because the flow fields agree while the boundary drifts. Third, independently moving objects cause the occlusion that breaks correspondence. A rejected pixel keeps its propagated label regardless: the check decides *when to re-infer*, not what to do with one bad pixel.

5.6 Threats to Validity

- **Quality rests on 70 evaluation points.** Quality is measured at ten frames per sequence while timing covers all 901. The comparison is paired, so the *difference* is well founded, but the sample is small. Early stopping also selected each checkpoint on these frames, so absolute mIoU is optimistic.
- **Single dataset, one altitude regime, one seed.** All pipeline results are UAVID; the flow-backend comparison is Ruralscapes, in a different propagation regime. Each backbone was trained once, so Table 5.1 supports a ranking but not a significance claim.
- **One hardware configuration.** Every timing comes from the same A100, and the speedup depends on the ratio of model cost to flow cost, so it would differ on an embedded accelerator.

6

Conclusions

6.1 Contributions

Temporal redundancy in UAV video is large enough to label every frame without running a segmentation model on every frame. Across the UAVid validation split the adaptive hybrid pipeline is $1.65\times$ faster than model-every-frame inference, retains 96.2% of the baseline mIoU and calls the model on 19.7% of frames, with per-sequence speedups between $1.30\times$ and $1.86\times$.

A single fixed threshold serves all seven sequences. The most dynamic, seq20, triggers 474 model calls and the calm seq37 only 87, so inference follows motion without per-sequence tuning, and the cost of motion is paid in speed rather than accuracy: seq20 falls to $1.30\times$ while its quality stays in line with the rest of the split.

What the threshold buys is quality, not speed. Flow is computed for every consecutive pair before the staleness decision, which makes flow the fixed cost and the model call the marginal one. Cutting model calls from 901 to 71 removes 92 % of the inference work yet yields only $1.9\times$, and going down to five calls buys almost no further speed while costing 0.20 mIoU.

The components were chosen by measurement. SEA-RAFT is $3.6\times$ faster per pair than FlowNet2 for a difference in propagation accuracy of 0.0006 mIoU, and running its correlation at quarter resolution is the largest single speed gain in the flow path, at a cost below 0.01 mIoU. Chunked streaming flow keeps peak GPU memory flat regardless of video length, against roughly 120 GB for naive precomputation, and backbones and flow backends remain interchangeable behind one configuration file.

The residual loss concentrates on small moving objects, principally **person** and **vehicle**: equal class weighting amplifies small regions, nearest-neighbour warping erodes their boundaries at every step, and independent object motion is where correspondence, and therefore the consistency check, is least reliable.

6.2 Lessons Learned

Measure the fixed cost, not the conditional one. Framing the work as reducing model calls measures the wrong variable: model calls fell by 92 % but runtime only by half, because flow is paid regardless. Parameter count was likewise a poor proxy for cost, since SEA-RAFT has far fewer parameters than the segmentation model yet needs roughly 20 GB for one native-resolution 4 K correlation volume.

A confidence signal is not a quality signal. The forward-backward check tests whether two flow fields agree geometrically. It is blind to the nearest-neighbour rounding that accumulates at label boundaries while flow stays self-consistent, which is why a frame can report high coverage and still show degraded small objects.

6.3 Future Work

- **Predict staleness without computing flow.** Flow is the fixed cost and is computed partly to decide whether to re-infer, so a cheap predictor from frame differences, IMU data or a low-resolution proxy would take flow off the decision path. This is the only route that attacks the cost bounding the system.
- **Overlap flow and inference on separate CUDA streams.** The pipeline runs them serially, yet a model call and the next chunk's flow are independent, so concurrent execution would raise throughput beyond what fewer model calls can reach.
- **Evaluate a smaller segmentation model** such as Hiera-Small. The achievable speedup depends on the ratio of model cost to flow cost, so a cheaper backbone changes the economics of the approach.
- **Cross-dataset evaluation.** Ruralscapes would test the method under altitude and parallax variation UAVid does not contain, and would allow the backend comparison to be re-run in the consecutive-frame regime the pipeline uses.



Pipeline Configuration

Table A.1: Adaptive pipeline configuration as used for all experiments.

Setting	Value	Note
backbone	<code>convnext_large</code>	any trained model
decoder	UPerNet	bilinear upsampling to input size
flow backend	<code>seaRAFT</code>	or <code>flowNet2</code>
model config	<code>spring-L</code>	12 recurrent refinements
scale	<code>-2</code>	quarter internal resolution
θ (<code>fb_threshold</code>)	1.5 px	round-trip error above which a pixel is invalid
K (<code>steps_per_chunk</code>)	16	32 pairs per launch
precision	<code>bf16</code>	
τ (<code>valid_threshold</code>)	0.95	re-infer at or below this coverage

B

Description of AI-Generated Content

As declared on the statement page, an AI assistant was used while preparing this report: Claude (Anthropic).

Language. The assistant checked grammar and spelling and proposed rephrasings where a passage was unclear or longer than it needed to be. The content, the claims and their interpretation are the author's.

Typesetting. It assisted with the $\text{\LaTeX}$ source: table layout and formatting, float placement, and drawing the TikZ diagrams and flowcharts to the author's specification.

Verification. The author reviewed and verified all AI-assisted content: every number against the result file it comes from, every figure against the data it plots, and the wording of every section against what the experiments actually show. The author retains full responsibility for the content of this report.

References

- [1] Z. Teed and J. Deng, “RAFT: Recurrent all-pairs field transforms for optical flow,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2020, pp. 402–419, best Paper Award.
- [2] Y. Wang, L. Lipson, and J. Deng, “SEA-RAFT: Simple, efficient, accurate RAFT for optical flow,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2024, pp. 36–54, oral, Best Paper Award candidate; arXiv:2405.14793.
- [3] Y. Lyu, G. Vosselman, G.-S. Xia, A. Yilmaz, and M. Y. Yang, “UAVid: A semantic segmentation dataset for UAV imagery,” *ISPRS Journal of Photogrammetry and Remote Sensing*, vol. 165, pp. 108–119, 2020.
- [4] A. Marcu, V. Licaret, D. Costea, and M. Leordeanu, “Semantics through time: Semi-supervised segmentation of aerial videos with iterative label propagation,” in *Proceedings of the Asian Conference on Computer Vision (ACCV)*, 2020, pp. 537–552, arXiv:2010.01910.
- [5] Y. Zhu, K. Sapra, F. A. Reda, K. J. Shih, S. Newsam, A. Tao, and B. Catanzaro, “Improving semantic segmentation via video propagation and label relaxation,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 8856–8865.
- [6] N. Sundaram, T. Brox, and K. Keutzer, “Dense point trajectories by GPU-accelerated large displacement optical flow,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2010, pp. 438–451.
- [7] T. Xiao, Y. Liu, B. Zhou, Y. Jiang, and J. Sun, “Unified perceptual parsing for scene understanding,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 418–434.
- [8] H. Zhao, J. Shi, X. Qi, X. Wang, and J. Jia, “Pyramid scene parsing network,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2881–2890.
- [9] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, “A ConvNet for the 2020s,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 11 976–11 986.
- [10] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, “Swin transformer: Hierarchical vision transformer using shifted windows,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 10 012–10 022.
- [11] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo, “SegFormer: Simple and efficient design for semantic segmentation with transformers,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 12 077–12 090.

-
- [12] C. Ryali, Y.-T. Hu, D. Bolya, C. Wei, H. Fan, P.-Y. Huang, V. Aggarwal, A. Chowdhury, O. Poursaeed, J. Hoffman, J. Malik, Y. Li, and C. Feichtenhofer, “Hiera: A hierarchical vision transformer without the bells-and-whistles,” in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 29 441–29 454.
 - [13] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Radle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. Dollar, and C. Feichtenhofer, “SAM 2: Segment anything in images and videos,” *arXiv preprint arXiv:2408.00714*, 2024.
 - [14] T. Brox, A. Bruhn, N. Papenberg, and J. Weickert, “High accuracy optical flow estimation based on a theory for warping,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, ser. Lecture Notes in Computer Science, vol. 3024. Springer, 2004, pp. 25–36.
 - [15] A. Dosovitskiy, P. Fischer, E. Ilg, P. Haeusser, C. Hazirbas, V. Golkov, P. van der Smagt, D. Cremers, and T. Brox, “FlowNet: Learning optical flow with convolutional networks,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 2758–2766.
 - [16] E. Ilg, N. Mayer, T. Saikia, M. Keuper, A. Dosovitskiy, and T. Brox, “FlowNet 2.0: Evolution of optical flow estimation with deep networks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2462–2470.
 - [17] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations (ICLR)*, 2019.